

AES-256 GCM File Encryption Utility

Production Guide for a Desktop Encryption Engine with Graphical User Interface

1. Project Overview

This system delivers a robust, cross-platform desktop framework designed to execute file-level encryption operations. Utilizing the modern AES-256-GCM symmetric encryption algorithm alongside a Tkinter graphical dashboard interface, the toolkit offers seamless, cryptographic preservation for locally cached assets.

2. Desktop Application Code Implementation (main.py)

Save the following implementation block into your project structure as **main.py**. Remember to initialize your virtual environment with the necessary dependencies via *pip install cryptography* beforehand:

```
import os
import tkinter as tk
from tkinter import filedialog, messagebox, ttk
from cryptography.hazmat.primitives.ciphers.aead import AESGCM
from cryptography.hazmat.primitives.kdf.scrypt import Scrypt

# Cryptographic Parameter Settings
SALT_SIZE = 16          # 128-bit salt for key derivation
NONCE_SIZE = 12         # 96-bit nonce optimal for AES-GCM mode
KEY_SIZE = 32           # 256-bit key size for AES-256
FILE_BUFFER = 64 * 1024 # 64KB chunks to manage memory efficiency safely

class EncryptorApp:
    def __init__(self, root):
        self.root = root
        self.root.title("AES-256 Cryptographic Shield")
        self.root.geometry("520x340")
        self.root.resizable(False, False)
        self.style = ttk.Style()
        self.style.theme_use("clam")
        self.setup_ui()

    def setup_ui(self):
        main_frame = ttk.Frame(self.root, padding="20")
        main_frame.pack(fill=tk.BOTH, expand=True)
        title_label = ttk.Label(main_frame, text="🛡️ AES-256 GCM File Guardian", font=("Segoe UI", 14, "bold"), foreground="#007ACC")
        title_label.pack(anchor=tk.W, pady=(0, 15))
```

```

        file_frame = ttk.LabelFrame(main_frame, text=" File Target
Configuration ", padding="10")
        file_frame.pack(fill=tk.X, pady=(0, 15))
        self.file_path_var = tk.StringVar()
        file_entry = ttk.Entry(file_frame, textvariable=self.file_path_var,
width=40)
        file_entry.pack(side=tk.LEFT, padx=(0, 10), expand=True, fill=tk.X)
        browse_btn = ttk.Button(file_frame, text="Browse...",
command=self.browse_file)
        browse_btn.pack(side=tk.RIGHT)
        pass_frame = ttk.LabelFrame(main_frame, text=" Security Key Management
", padding="10")
        pass_frame.pack(fill=tk.X, pady=(0, 20))
        ttk.Label(pass_frame, text="Master Secret
Password:").pack(anchor=tk.W, pady=(0, 5))
        self.password_var = tk.StringVar()
        self.password_entry = ttk.Entry(pass_frame,
textvariable=self.password_var, show="•", width=50)
        self.password_entry.pack(fill=tk.X, expand=True)
        btn_frame = ttk.Frame(main_frame)
        btn_frame.pack(fill=tk.X)
        self.encrypt_btn = ttk.Button(btn_frame, text="🔒 Encrypt File",
command=self.process_encryption)
        self.encrypt_btn.pack(side=tk.LEFT, fill=tk.X, expand=True, padx=(0,
10))
        self.decrypt_btn = ttk.Button(btn_frame, text="🔓 Decrypt File",
command=self.process_decryption)
        self.decrypt_btn.pack(side=tk.RIGHT, fill=tk.X, expand=True, padx=(10,
0))

        self.progress_bar = ttk.Progressbar(main_frame, mode='determinate')
        self.progress_bar.pack(fill=tk.X, pady=(15, 0))

    def browse_file(self):
        filename = filedialog.askopenfilename(title="Select Target File")
        if filename: self.file_path_var.set(filename)

    def derive_key(self, password: bytes, salt: bytes) -> bytes:
        kdf = Scrypt(salt=salt, length=KEY_SIZE, n=2**14, r=8, p=1)
        return kdf.derive(password)

    def process_encryption(self):
        file_path = self.file_path_var.get()
        password = self.password_var.get()
        if not file_path or not os.path.exists(file_path):
            messagebox.showerror("Error", "Please select a valid file path
target.")
            return
        if not password:
            messagebox.showerror("Error", "A master security password is

```

```

required.")
    return
    output_path = file_path + ".enc"
    try:
        salt = os.urandom(SALT_SIZE)
        nonce = os.urandom(NONCE_SIZE)
        key = self.derive_key(password.encode(), salt)
        aesgcm = AESGCM(key)
        self.progress_bar['value'] = 10
        self.root.update_idletasks()
        with open(file_path, 'rb') as f_in: plaintext = f_in.read()
        ciphertext = aesgcm.encrypt(nonce, plaintext, None)
        with open(output_path, 'wb') as f_out:
            f_out.write(salt); f_out.write(nonce); f_out.write(ciphertext)
        self.progress_bar['value'] = 100
        messagebox.showinfo("Success", f"File successfully
protected!\nSaved to: {os.path.basename(output_path)}")
        self.clear_fields()
    except Exception as e:
        messagebox.showerror("Cryptographic Fault", f"Encryption failed:
{str(e)}")
    finally: self.progress_bar['value'] = 0

def process_decryption(self):
    file_path = self.file_path_var.get()
    password = self.password_var.get()
    if not file_path or not os.path.exists(file_path):
        messagebox.showerror("Error", "Please select a valid file
target.")
    return
    if not password:
        messagebox.showerror("Error", "Master password input missing.")
    return
    output_path = file_path.rsplit('.enc', 1)[0]
    if output_path == file_path: output_path = file_path + ".dec"
    try:
        self.progress_bar['value'] = 10
        self.root.update_idletasks()
        with open(file_path, 'rb') as f_in:
            salt = f_in.read(SALT_SIZE)
            nonce = f_in.read(NONCE_SIZE)
            ciphertext = f_in.read()
        key = self.derive_key(password.encode(), salt)
        aesgcm = AESGCM(key)
        plaintext = aesgcm.decrypt(nonce, ciphertext, None)
        with open(output_path, 'wb') as f_out: f_out.write(plaintext)
        self.progress_bar['value'] = 100
        messagebox.showinfo("Success", f"Integrity match
verified!\nDecrypted: {os.path.basename(output_path)}")

```

```

        self.clear_fields()
    except Exception:
        messagebox.showerror("Verification Failure", "Decryption failed!
        Password invalid or file corrupted.")
    finally: self.progress_bar['value'] = 0

    def clear_fields(self):
        self.file_path_var.set("");
        self.password_var.set("")

if __name__ == "__main__":
    root = tk.Tk()
    app = EncryptorApp(root)
    root.mainloop()

```

3. Core Cryptographic Architectures

- **AES-256-GCM Authenticated Structure:** Unlike common legacy operating profiles like AES-CBC, Galois/Counter Mode handles authenticating payloads implicitly. If an adversary alters a bit within the encrypted blob structure, the tag engine breaks, rejecting downstream mutation.
- **Script Key Derivation Protocols:** To counter generic brute-force mechanisms targeting plain text user credentials, inputs are transformed by a Script layer operating against a 128-bit randomized salt sequence. This structural model prevents hyper-parallelized GPU exploitation.
- **Cryptographic Nonce Initialization Hygiene:** Every runtime invocation builds an independent 96-bit initialization vector extracted via system-level hardware entropy streams. This behavior patterns distinct output files even if uniform content is evaluated sequentially under matching keys.